VL2Decision: Typed Decision Models for Self-Improving Embodied Agents

Jaehoon Kim, Kwangwook Seo, Jeongeun Lee, Sunghwan Kim, Serin Kim, Dongha Lee*
Yonsei University, Seoul, Republic of Korea
jaeh8nkim@yonsei.ac.kr, donalee@yonsei.ac.kr
*Corresponding author

***Abstract*—Embodied agents need to use experiential knowledge from past executions to improve later decisions. Existing approaches often leave unclear which underlying decisions need correction after a failed attempt. We propose VL2Decision, which uses a Typed Decision Model (TDM) to make these decisions explicit. A TDM uses a vision-language model (VLM) to answer typed questions using supplied knowledge. Each question yields a probability distribution over fixed options through one next-token prediction, avoiding the generation and parsing of action descriptions. The records link each question and its full answer distribution to the corresponding frame. An LLM optimizer analyzes these records and task outcomes to refine this knowledge. We evaluate in Minecraft, an embodied control testbed, on 46 tasks not used to develop this knowledge. TDMs reach 79% success with a proprietary VLM, 76% with an open 27B VLM, and 53% with an open 9B VLM. STEVE-1 and JARVIS-VLA reach 29% and 25%, respectively. Giving JARVIS-VLA the same experiential knowledge yields little improvement. VL2Decision enables embodied agents to improve by acquiring and applying experiential knowledge.**

***Index Terms*—embodied agents, vision-language-action models, typed decision models, self-improving agents**

I. Introduction

Hierarchical embodied agents use an LLM planner to choose subgoals and a controller to act on visual input [1], [2]. Analyzing past attempts can reveal how to execute a subgoal more reliably [3]. We call these reusable lessons experiential knowledge. This knowledge helps only if the controller can use it in later decisions. Controllers based on visuomotor policies, which map visual inputs to actions [4], or vision-language models (VLMs) [5] are fine-tuned for action prediction. Such fine-tuning can weaken a VLM's language understanding and reasoning abilities [6] needed to apply newly supplied knowledge. Moreover, their action outputs alone do not identify which underlying decision to revise when execution fails. We study how to make these decisions explicit and improve them using experiential knowledge.

General-purpose VLMs can use supplied knowledge to generate formatted action text [7] or executable code [8], which must be decoded token by token and then parsed or executed. Their outputs can include explanations, yet do not directly provide the probability assigned to each available answer at the time of decision. For example, a failed interaction may stem from mistakenly deciding that a target is within reach or from choosing an unsuitable action; these errors call for different fixes. We propose VL2Decision (Vision-Language to Decision), which uses a Typed Decision Model (TDM) as the controller. A TDM answers typed questions about what it sees and what to do, returning a distribution over fixed options for each question. This uses the VLM's visual question answering ability [9] in place of a policy trained to predict actions. The questions specify what must be decided, while **the supplied knowledge can explain how to decide it in the current situation.** Each distribution is read from next-token logits, so a question takes one token to answer rather than a sequence of tokens describing an action.

The resulting record ties each question and its full answer distribution to the frame at which the decision was made. Even when the chosen action is the same, these records show whether an answer had high probability or was close to alternatives. Prompt optimization uses execution feedback to revise instructions [10]. Here, an LLM optimizer **compares these records with the observed scene and task outcomes to identify which decisions need correction.** It expresses the resulting experiential knowledge in the domain instructions and question wording. For instance, it can refine what counts as a reachable target and inspect how decisions about reach change in later runs. The revised text supplies this knowledge in subsequent rollouts, allowing the same correction to guide decisions in different scenes. We evaluate VL2Decision in Minecraft, a visual testbed for embodied control, under a shared planner on tasks not used to develop this knowledge. TDMs with experiential knowledge outperform the fine-tuned controllers STEVE-1 and JARVIS-VLA with both proprietary and open VLMs. We also test whether a fine-tuned controller benefits from the same knowledge: JARVIS-VLA shows little improvement when given the TDM's instructions. Our contributions are the TDM interface, a process for deriving experiential knowledge from decision logs, and an evaluation on held-out tasks.

II. Method

Planner: A shared planner (GPT-6 Luna) generates subgoals in each controller's input format. The TDM receives the subgoal as written; STEVE-1 receives a short text instruction, limited by its encoder to 77 tokens; JARVIS-VLA receives an atomic-task key such as `mine_block:oak_log`, mapped to a standard instruction. We also test JARVIS-VLA with the text supplied to the TDM before and after adding experiential knowledge, in place of the key.

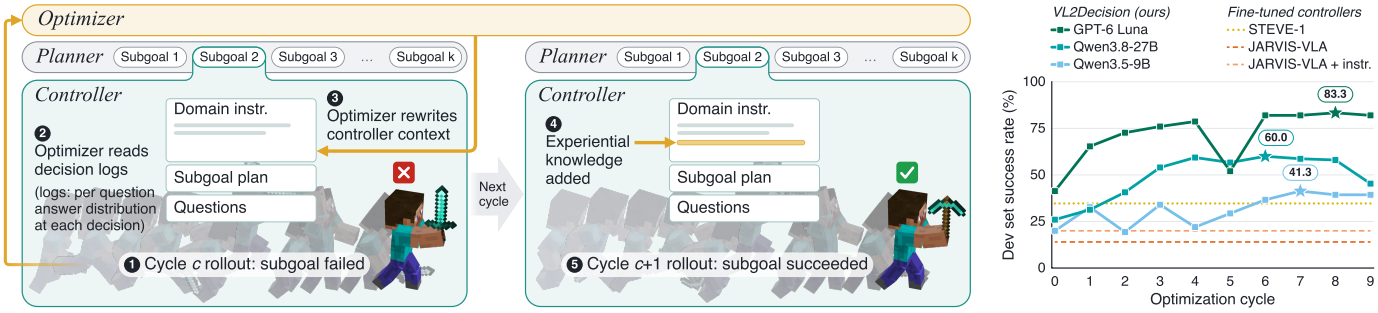


Fig. 1. Iterative self-improvement. Left: two optimization cycles. Right: dev set success rate per cycle; cycle 0 and stars: versions run on the test set (Table I).

TABLE I
SUCCESSFUL RUNS ON THE TEST SET (46 MCU TASKS, FIVE SEEDS)

Controller	(25)	(60)	(20)	(25)	(70)	(30)	SR↑ (230)	Ticks↓	Time↓
Fine-tuned controllers									
STEVE-1	23	20	0	5	15	4	29%	68	**0.11 s**
JARVIS-VLA	18	27	4	1	5	2	25%	171	1.21 s
JARVIS-VLA + instr.[a]	15	29	2	0	9	6	27%	142	1.30 s
VL2Decision (ours), initial									
GPT-6 Luna	16	30	0	15	40	14	50%	174	2.24 s
Qwen3.8-27B	7	12	0	14	36	6	33%	225	2.94 s
Qwen3.5-9B	12	8	1	13	35	1	30%	186	1.49 s
VL2Decision (ours), with experiential knowledge									
GPT-6 Luna	**25**	**40**	**13**	**16**	58	**30**	**79%**	62	2.17 s
Qwen3.8-27B	24	31	11	**16**	**62**	**30**	76%	**44**	3.52 s
Qwen3.5-9B	22	17	0	15	47	22	53%	66	1.76 s

Columns, left to right: gather, mine, hunt, place and move, use an item, interact (runs), success rate, median ticks to success, time per game second. [a]Given the GPT-6 Luna TDM's domain instruction with experiential knowledge.

Controllers: The TDM decides every 4 ticks; the fine-tuned controllers act every tick using their native image formats (20 ticks/s). Its input contains the domain instruction, subgoal plan and progress, the current frame, and up to eight past frames. It answers 18 questions: five on perception, including target visibility and reach, and 13 on action, including camera rotation and attack. We use GPT-6 Luna, Qwen3.8-27B or Qwen3.5-9B as the TDM; JARVIS-VLA uses Qwen2-VL-7B, and STEVE-1 has 0.31B parameters.

Iterative Self-Improvement: Each cycle, an optimizer (GPT-6 Astra) compares subgoal outcomes, frames and question distributions to identify decisions that need correction (Fig. 1). It expresses the resulting experiential knowledge in the TDM's domain instruction and question wording for subsequent runs. Question types, options and model weights remain fixed.

III. EXPERIMENTS AND RESULTS

Setup: The development (dev) set contains 30 atomic tasks, such as collecting sand, killing a pig and planting wheat seeds. The held-out test set contains 46 short-horizon, programmatically verifiable MCU tasks [11], such as leading a cow, sleeping in a bed and lighting a fire. Each controller runs once per task and world seed, using five seeds. Both sets are evaluation-only for the fine-tuned controllers. We derive experiential knowledge only from dev runs.

Main Results: TDMs with experiential knowledge outperform both fine-tuned controllers under the same planner model (Table I; Fig. 1). Qwen3.8-27B nearly matches GPT-6 Luna, and Qwen3.5-9B remains above both baselines. Giving JARVIS-VLA the TDM's instructions, either before or after adding experiential knowledge, yields little improvement.

Execution Improvements: Execution improves as the TDM incorporates experiential knowledge. Initially, TDMs lead on item use and placement and movement, but mostly trail on gathering and mining. After adding this knowledge, the gap narrows and median ticks to success fall from 174–225 to 44–66, below the fine-tuned controllers' 68–171, among successful runs. For example, the agent had to shear a mooshroom, a cow with mushrooms on its back, and collect the mushrooms. The TDM assigned 0.67–0.95 probability to the animal being within reach, although the crosshair was on the mushrooms above its body and repeated shearing had no effect. The optimizer clarified that shearing requires aiming at the body. The revised instruction guides later reach decisions. **The logs identify which decisions need more precise instructions.**

Adaptation Cost: Starting from pretrained models, deriving experiential knowledge costs under $100 with GPT-6 Luna, optimizer included, or 30–42 B200 GPU-hours plus optimizer API calls with open VLMs. STEVE-1 reports $60 for fine-tuning, excluding VPT pretraining [4], [12]; JARVIS-VLA reports 640 A800 GPU-hours of post-training [5]. Both use human gameplay from a dataset collected for about $90k [12]. Our process derives experiential knowledge from execution feedback without collecting action demonstrations.

IV. CONCLUSION

We present VL2Decision, which connects experiential knowledge to subsequent control through typed decisions. Each recorded question and answer distribution can be inspected against its frame, allowing an LLM optimizer to formulate experiential knowledge for later decisions. Experiments in Minecraft show improvements on held-out tasks with both proprietary and open VLMs. The limited gains from supplying the same knowledge to JARVIS-VLA suggest that effective self-improvement requires both deriving knowledge and a decision process that can apply it. Extending this approach to longer-horizon tasks and other embodied environments remains future work.

REFERENCES

[1] Z. Wang *et al.*, "Describe, explain, plan and select: Interactive planning with llms enables open-world multi-task agents," in *Advances in Neural Information Processing Systems*, A. Oh *et al.*, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 34 153–34 189.

[2] L. X. Shi *et al.*, "Hi robot: Open-ended instruction following with hierarchical vision-language-action models," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Singh *et al.*, Eds., vol. 267. PMLR, 13–19 Jul 2025, pp. 54 919–54 933.

[3] N. Shinn *et al.*, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, A. Oh *et al.*, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 8634–8652.

[4] S. Lifshitz *et al.*, "Steve-1: A generative model for text-to-behavior in minecraft," in *Advances in Neural Information Processing Systems*, A. Oh *et al.*, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 69 900–69 929.

[5] M. Li *et al.*, "JARVIS-VLA: Post-training large-scale vision language models to play visual games with keyboards and mouse," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che *et al.*, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 17 878–17 899.

[6] A. Hancock *et al.*, "Actions as language: Fine-tuning vlms into vlas without catastrophic forgetting," in *International Conference on Learning Representations*, C. Vondrick *et al.*, Eds., vol. 2026, 2026, pp. 75 333–75 360.

[7] T. Ju *et al.*, "MineExplorer: Evaluating open-world exploration of MLLM agents in Minecraft," in *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, H.-Y. Lee *et al.*, Eds. Budapest, Hungary: Association for Computational Linguistics, Oct. 2026, pp. 8542–8570.

[8] W. Tan *et al.*, "Cradle: Empowering foundation agents towards general computer control," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Singh *et al.*, Eds., vol. 267. PMLR, 13–19 Jul 2025, pp. 58 658–58 725.

[9] S. Nasiriany *et al.*, "PIVOT: Iterative visual prompting elicits actionable knowledge for VLMs," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov *et al.*, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 37 321–37 341.

[10] L. A. Agrawal *et al.*, "Gepa: Reflective prompt evolution can outperform reinforcement learning," in *International Conference on Learning Representations*, C. Vondrick *et al.*, Eds., vol. 2026, 2026, pp. 8479–8565.

[11] X. Zheng *et al.*, "MCU: An evaluation framework for open-ended game agents," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Singh *et al.*, Eds., vol. 267. PMLR, 13–19 Jul 2025, pp. 78 221–78 259.

[12] B. Baker *et al.*, "Video pretraining (vpt): Learning to act by watching unlabeled online videos," in *Advances in Neural Information Processing Systems*, S. Koyejo *et al.*, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 24 639–24 654.